



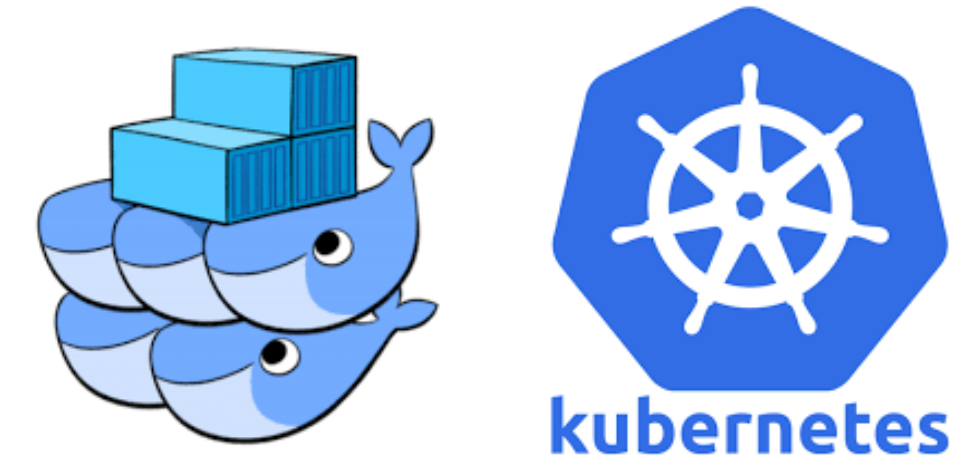
DR B. DIOP

babacar.diop@ussein.edu.sn

Conteneurisation & orchestration

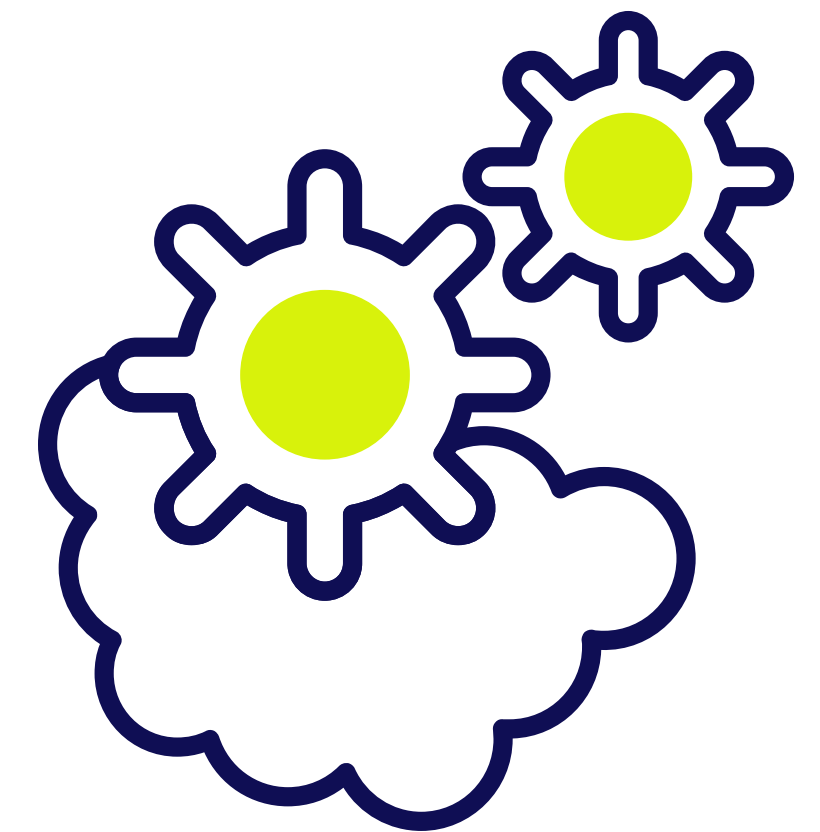
Kubernetes Orchestration

APIs Kubernetes



INFO – AGROTIC

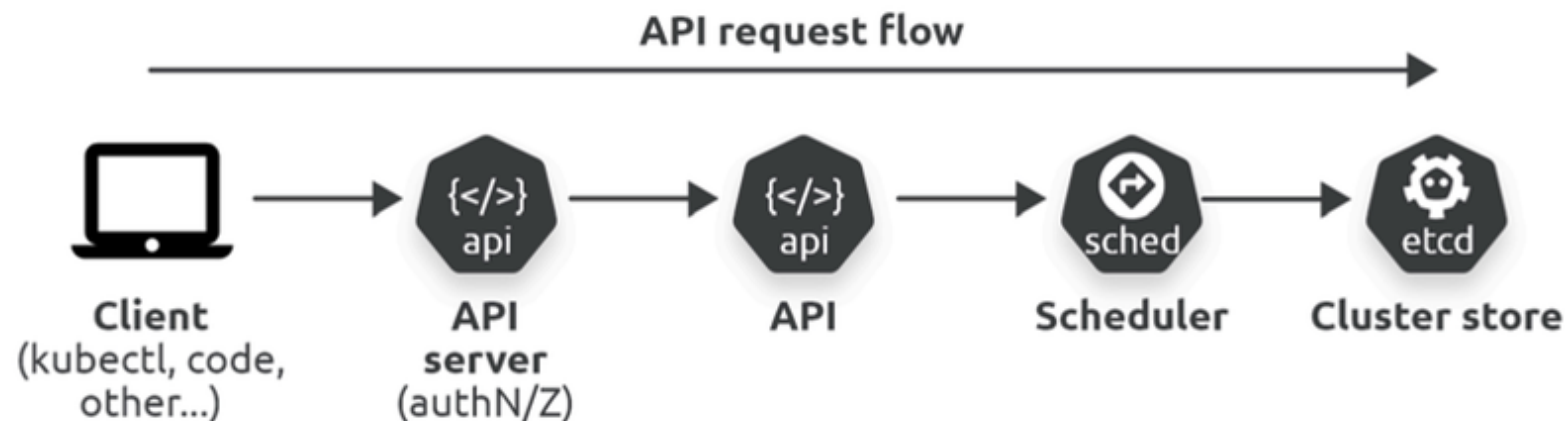
Last update: dec. 2025



Vue d'ensemble de l'API Kubernetes

Les clients envoient des requêtes à Kubernetes pour créer, lire, mettre à jour ou supprimer des objets (**Pods**, **Services**, etc.).

- Outils principaux : **kubectl**, **code applicatif**, **outils de test d'API**.
- Point central : Toutes les requêtes passent par l'**API Server** pour **auth** et **autorisation**.
- Si validées : Exécution sur le **cluster** et persistance de l'état dans le **cluster store** (etcd).



L'API Server de Kubernetes

Rôle principal : Exposer l'**API Kubernetes** via une interface RESTful sécurisée (HTTPS).

Fonction : Point central de communication ("Grand Central") pour tous les composants Kubernetes.

Exemples d'interactions :

- **kubectl** envoie toutes les commandes au **serveur API**.
- Les nœuds (**Kubelets**) surveillent le **serveur API** pour de nouvelles tâches.
- Les services du plan de contrôle (control plane) communiquent via le **serveur API**.

Architecture et déploiement

Localisation : Service du plan de contrôle, exécuté comme **Pods** dans le namespace **kube-system**.

Disponibilité et performance :

- Nécessite une haute disponibilité et des performances adaptées.
- Géré par le fournisseur de cluster dans les solutions hébergées.
- Sécurité : Utilise TLS, authentification et autorisation pour valider les requêtes.

API RESTful et méthodes HTTP

Style d'API : **RESTful (REpresentational State Transfer)**.

Opérations CRUD : **Create, Read, Update, Delete** via des méthodes HTTP standard.

Méthode HTTP	Verbe CRUD	Commande <code>kubectl</code>
POST	create	<code>kubectl create -f <fichier></code>
GET	get/list	<code>kubectl get pods</code>
PUT/PATCH	update	<code>kubectl edit deployment</code>
DELETE	delete	<code>kubectl delete ingress</code>

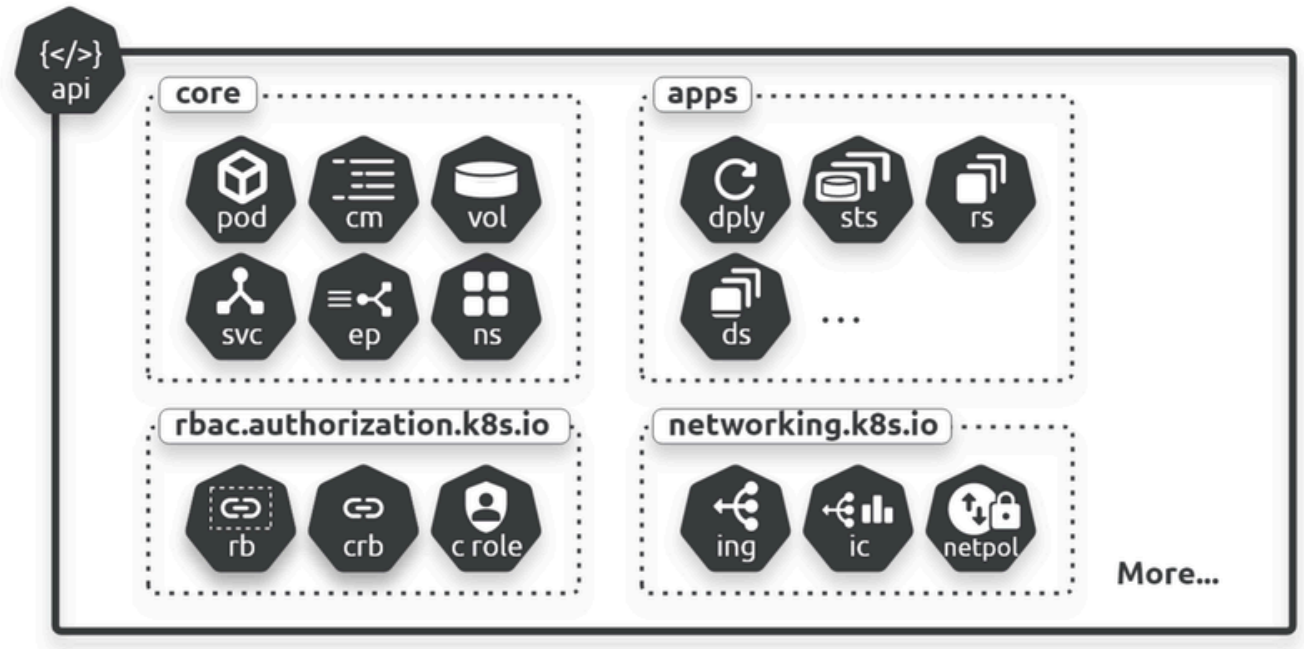
Ports courants : 443 ou 6443 (configurable).

Groupes d'API sur Kubernetes

L'image présente l'API avec **4 groupes** (Il y en a bien plus).

Au niveau le plus élevé, il existe deux types de groupes d'API :

- Le groupe principal
- Les groupes nommés



Chemins d'API et groupes

Structure des chemins REST :

- Groupe de base (core) : **/api/v1/** (ex. : Pods, Services, Namespaces).
- Groupes nommés : **/apis/<nom-groupe>/<version>/**
 - (ex. : **apps/v1, networking.k8s.io/v1**).

Ressources avec/sans espace de noms :

- Avec espace de noms : **/api/v1/namespaces/{namespace}/pods/**
- Sans espace de noms : **/api/v1/nodes/**

Vue d'ensemble du groupe d'API Core

Qu'est-ce que c'est ?

- Ressources d'**API matures** des **premiers jours de Kubernetes**
- Accessible via le chemin **/api/v1/**
- Contient les **objets Kubernetes fondamentaux et essentiels**

Caractéristiques clés :

- Créées avant la mise en place des **groupes d'API**
- Considérées comme des **composants fondamentaux "core"**
- Aucune nouvelle ressource n'est généralement ajoutée à ce groupe

Exemples de ressources du groupe Core

Ressources Core courantes :

Type de ressource	Exemple de chemin API
Pods	<code>/api/v1/namespaces/{namespace}/pods/</code>
Services	<code>/api/v1/namespaces/{namespace}/services/</code>
Nodes	<code>/api/v1/nodes/</code>
Namespaces	<code>/api/v1/namespaces/</code>
Secrets	<code>/api/v1/namespaces/{namespace}/secrets/</code>
ServiceAccounts	<code>/api/v1/namespaces/{namespace}/serviceaccounts/</code>

Ressources avec namespace vs sans namespace

Ressources avec namespace :

- Doivent exister dans un **namespace spécifique**
- Le chemin inclut le **segment du namespace**
- Exemple : **/api/v1/namespaces/shield/pods/**
- Autres exemples : **Pods, Services, Secrets, ConfigMaps**

Ressources sans namespace (cluster-wide) :

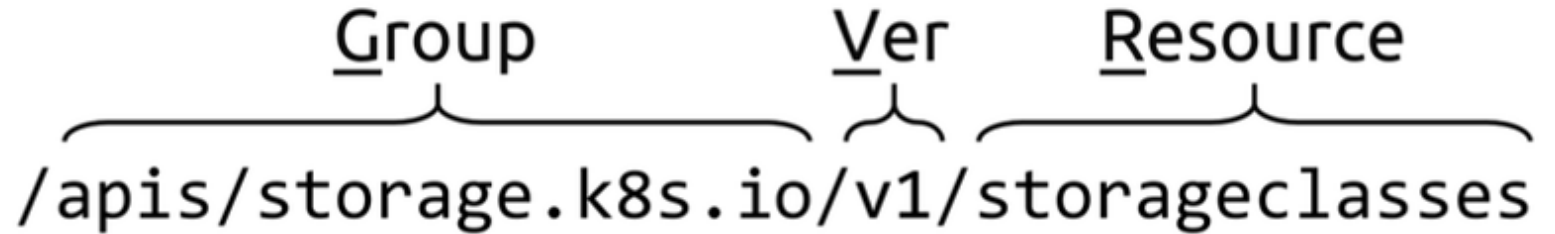
- Existent au niveau du cluster, hors namespace
- Exemple : **/api/v1/nodes/**
- Autres exemples : **Nodes, PersistentVolumes, StorageClasses**

Structure GVR (Group, Version, Resource)

GVR - Acronyme pour comprendre les chemins API :

- **Groupe** : v1 (pour le groupe Core)
- **Version** : v1
- **Ressource** : pods, services, nodes, etc.

Structure typique :


/apis/storage.k8s.io/v1/storageclasses

Points clés à retenir

1. **Groupe Core = /api/v1/**

- API singleton (pas de nom de groupe spécifique)
- Ressources fondamentales et matures

2. **Deux types de portée :**

- Avec namespace : chemin plus long
- Sans namespace : chemin direct

3. **Structure cohérente :**

- Suit le modèle GVR
- Les chemins sont prévisibles

4. **Important pour :**

- Les opérations quotidiennes avec kubectl
- Le développement d'applications
- L'automatisation via l'API directe

Vue d'ensemble des groupes d'API nommés

Groupes d'API nommés - L'avenir de l'API Kubernetes

Définition :

- Groupes spécialisés organisés par domaine fonctionnel
- Contiennent les ressources créées après la structuration de l'API
- Chemin standard : **/apis/{nom-groupe}/{version}/**

Positionnement :

- Toutes les **nouvelles ressources** vont dans des groupes nommés
- Parfois appelés "sous-groupes" (**sub-groups**)
- Alternative au groupe "**core**" historique

Organisation par domaines fonctionnels

Groupes par catégorie métier

apps (Applications) :

- Gestion des charges de travail
- Exemples : **Deployments, ReplicaSets, DaemonSets, StatefulSets**

networking.k8s.io (Réseau) :

- Ressources réseau avancées
- Exemples : **Ingresses, Ingress Classes, Network Policies**

rbac.authorization.k8s.io (RBAC) :

- Contrôle d'accès basé sur les rôles
- Exemples : **Roles, RoleBindings, ClusterRoles, ClusterRoleBindings**

storage.k8s.io (Stockage) :

- Gestion du stockage persistant
- Exemples : **StorageClasses, CSIDrivers, VolumeAttachments**

Exemples concrets de chemins d'API

Tableau comparatif des chemins

Ressource	Groupe	Chemin d'API complet
Ingress	networking.k8s.io	<code>/apis/networking.k8s.io/v1/namespaces/{namespace}/ingresses/</code>
RoleBinding	rbac.authorization.k8s.io	<code>/apis/rbac.authorization.k8s.io/v1/namespaces/{namespace}/rolebindings/</code>
ClusterRole	rbac.authorization.k8s.io	<code>/apis/rbac.authorization.k8s.io/v1/clusterroles/</code>
StorageClass	storage.k8s.io	<code>/apis/storage.k8s.io/v1/storageclasses/</code>
Deployment	apps	<code>/apis/apps/v1/namespaces/{namespace}/deployments/</code>

Différences avec le groupe Core

Comparaison Core vs Groupes nommés

Groupe Core (/api/v1/) :

- Historique, ressources fondamentales
- Pas de nom de groupe spécifique
- Exemples : **Pods, Services, Nodes, Namespaces**
- Structure : **/api/v1/[ressource]**

Groupes nommés (/apis/.../) :

- Ressources spécialisées plus récentes
- Nom de groupe explicite dans le chemin
- Exemples : **Deployments, Ingresses, StorageClasses**
- Structure : **/apis/{groupe}/{version}/[ressource]**

Note : Les **Pods** et **Services** seraient probablement dans **apps** et **networking.k8s.io** s'ils étaient créés aujourd'hui.

Avantages de la modularisation

Pourquoi des groupes nommés ?

Scalabilité :

- API plus facile à maintenir et étendre
- Évite la "monolithisation" de l'API

Navigabilité :

- Organisation logique par domaines
- Facilite la découverte des ressources

Extensibilité :

- Permet d'ajouter facilement de nouvelles fonctionnalités
- Supporte mieux les API personnalisées (CRD)

Gestion des versions :

- Versionnement indépendant par groupe
- Évolution plus flexible

Commandes d'exploration pratiques

Outils pour explorer l'API

Liste toutes les ressources disponibles :

```
kubectl api-resources
```

- Affiche : noms, groupes, versions, shortnames
- Indique si les ressources sont namespaced ou cluster-scoped

Exemple de sortie :

NAME	SHORTNAMES	APIGROUP	NAMESPACED	KIND
deployments	deploy	apps	true	Deployment
ingresses	ing	networking.k8s.io	true	Ingress
storageclasses	sc	storage.k8s.io	false	StorageClass
clusterroles		rbac.authorization.k8s.io	false	ClusterRole

Commandes d'exploration pratiques

Outils pour explorer l'API

Liste toutes les ressources disponibles :

```
kubectl api-resources
```

- Affiche : noms, groupes, versions, shortnames
- Indique si les ressources sont namespaced ou cluster-scoped

Exemple de sortie :

NAME	SHORTNAMES	APIGROUP	NAMESPACED	KIND
deployments	deploy	apps	true	Deployment
ingresses	ing	networking.k8s.io	true	Ingress
storageclasses	sc	storage.k8s.io	false	StorageClass
clusterroles		rbac.authorization.k8s.io	false	ClusterRole

Commandes d'exploration pratiques

Outils pour explorer l'API

Liste toutes les ressources disponibles :

```
kubectl api-resources
```

- Affiche : noms, groupes, versions, shortnames
- Indique si les ressources sont namespaced ou cluster-scoped

Liste des versions d'API supportées :

```
kubectl api-versions
```

Accès direct à l'API Kubernetes

Au-delà de **kubectl** : Explorer l'API directement

Pourquoi explorer l'API directement ?

- Compréhension plus profonde du fonctionnement interne
- Débogage et investigation avancée
- Automatisation et scripts personnalisés

Options d'accès direct :

1. Outils de développement d'API (**Postman, Insomnia**, etc.)
2. Commandes HTTP (**curl, wget, Invoke-WebRequest**)
3. Navigateur web pour la navigation manuelle

kubectl proxy - La méthode simplifiée

Proxy d'API avec **kubectl**

Commande de base : `kubectl proxy --port 9000 &`

- `--port 9000` : Expose sur le port spécifié
- `&` : Exécute en arrière-plan (background)

Fonctionnalités du proxy :

- Expose l'API sur l'interface localhost
- Gère automatiquement l'authentification et la sécurité
- Pas besoin de configurer manuellement les certificats TLS

Exploration de la structure de l'API

Découverte des groupes et versions d'API

Liste des versions du **groupe Core** :

```
curl http://localhost:9000/api
```

Réponse JSON :

```
{  
  "kind": "APIVersions",  
  "versions": ["v1"],  
  "serverAddressByClientCIDRs": [...]  
}
```

Exploration de la structure de l'API

Découverte des groupes et versions d'API

Liste de tous les **groupes nommés** :

Réponse JSON :

```
curl http://localhost:9000/apis
```

```
{
  "kind": "APIGroupList",
  "apiVersion": "v1",
  "groups": [
    {
      "name": "apps",
      "versions": [{
        "groupVersion": "apps/v1",
        "version": "v1"
      }],
      "preferredVersion": {
        "groupVersion": "apps/v1",
        "version": "v1"
      }
    }
  ]
}
```


Exemples pratiques de requêtes

1. Lister tous les Pods dans un namespace :

```
GET /api/v1/namespaces/shield/pods/
```

Code réponse attendu : 200 (OK) ou 401 (Non autorisé)

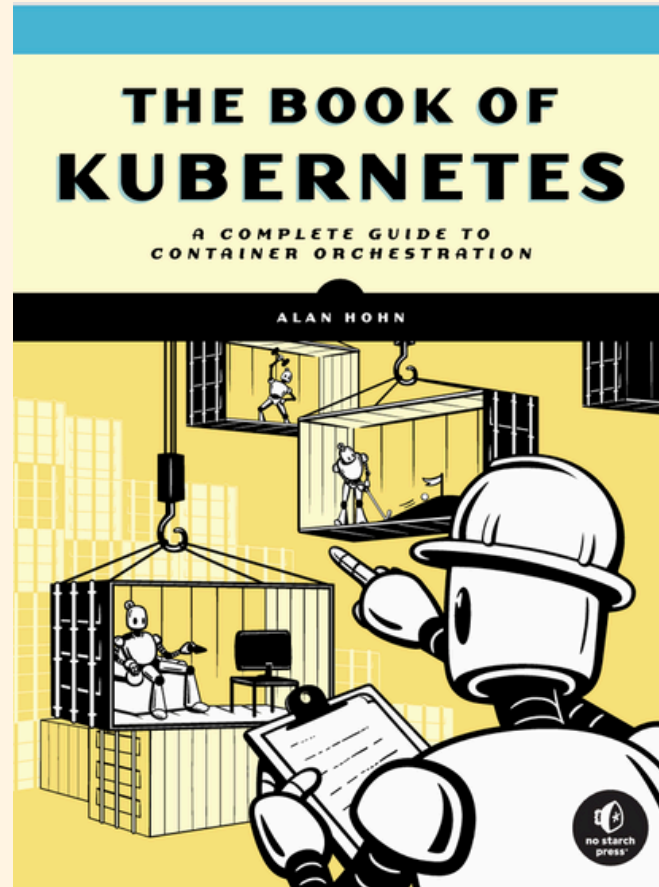
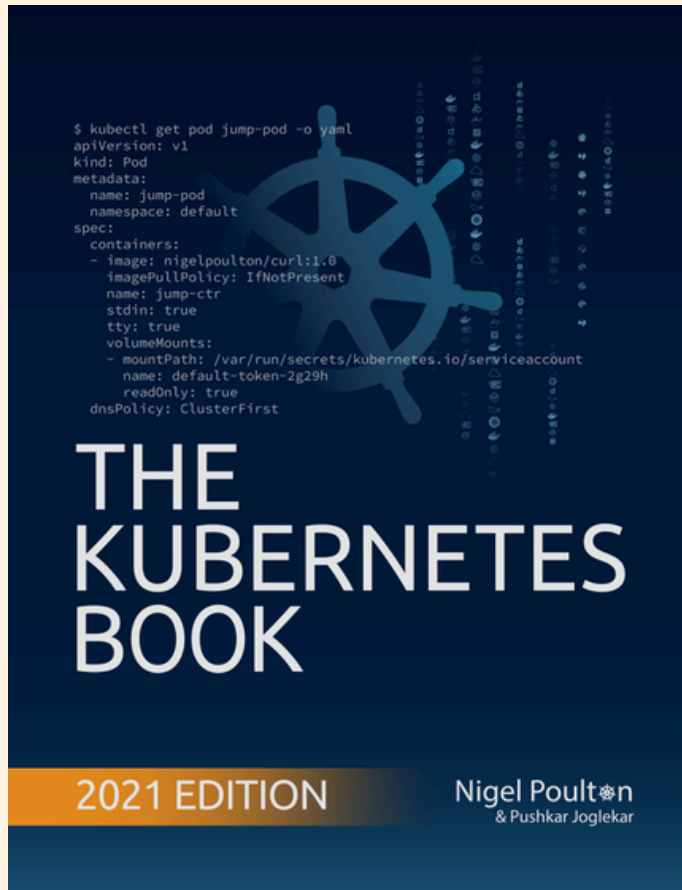
2. Lister tous les Nodes du cluster :

```
GET /api/v1/nodes/
```

3. Créer un nouveau namespace :

```
POST /api/v1/namespaces/
```

Documentation



Kubernetes Official Website Documentation

Kubernetes Documentation

Kubernetes is an open source container orchestration engine for automating deployment, scaling, and management of containerized applications. The open source project is hosted by the Cloud Native...

